DevOps CI/CD Pipeline Implementation using Jenkins, Docker, GitHub and AWS EC2

Prepared By:
Sharath Koppula

Domain: DevOps Engineering

---

1. Project Overview

This project demonstrates the implementation of a DevOps Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins, Docker, GitHub, and AWS EC2.

The pipeline automates the process of retrieving application code from GitHub, building a Docker image, and deploying the application container on an AWS EC2 server.

The main objective of this project is to simulate a real-world DevOps automation workflow used in modern software development environments.

---

2. Tools & Technologies Used

Tool	Purpose
GitHub	Source code repository
Jenkins	CI/CD automation
Docker	Containerization
AWS EC2	Cloud server hosting
Node.js	Web application runtime
Linux	Server operating system

Technologies used in the project:

- Jenkins
- Docker
- GitHub
- Amazon Web Services

---

3. System Architecture

Developer
|
▼
GitHub Repository
|
▼
Jenkins CI/CD Pipeline
|
▼
Docker Image Build
|
▼
Docker Container Deployment
|
▼
AWS EC2 Server
|
▼
Running Web Application

---

4. GitHub Repository

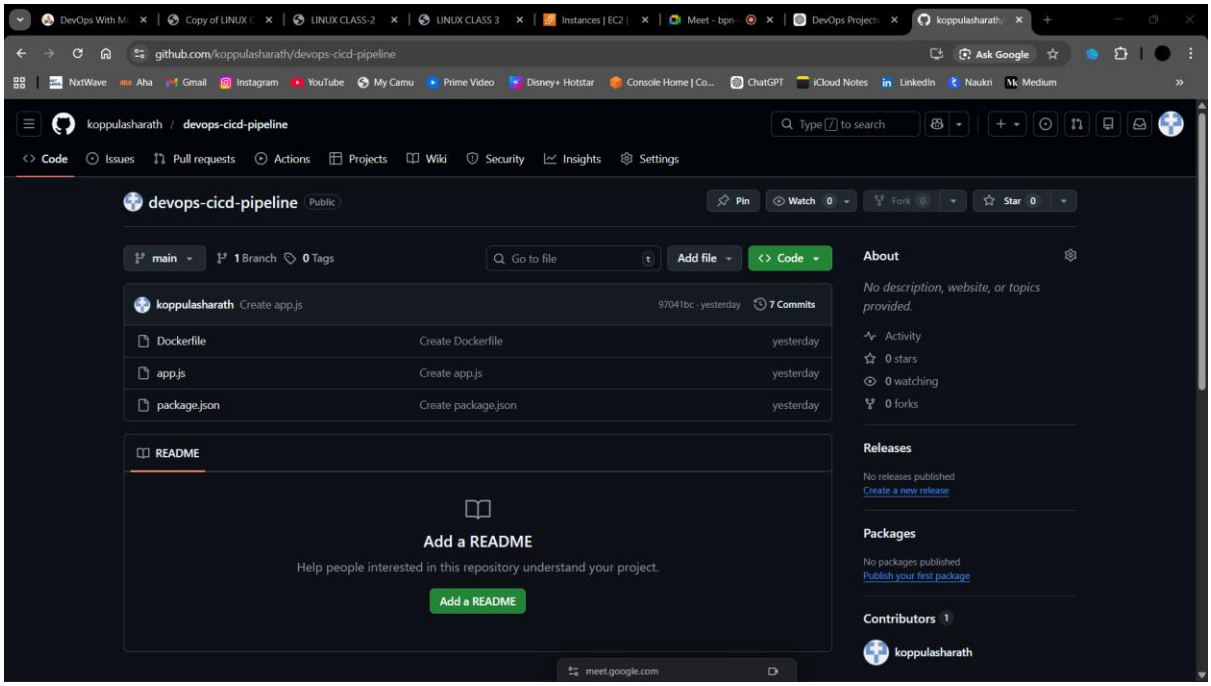
Repository URL:

https://github.com/koppulasharath/devops-cicd-pipeline

Repository contains the following files:

Dockerfile
app.js
package.json

These files are used by Jenkins to build the Docker image and deploy the application.



---

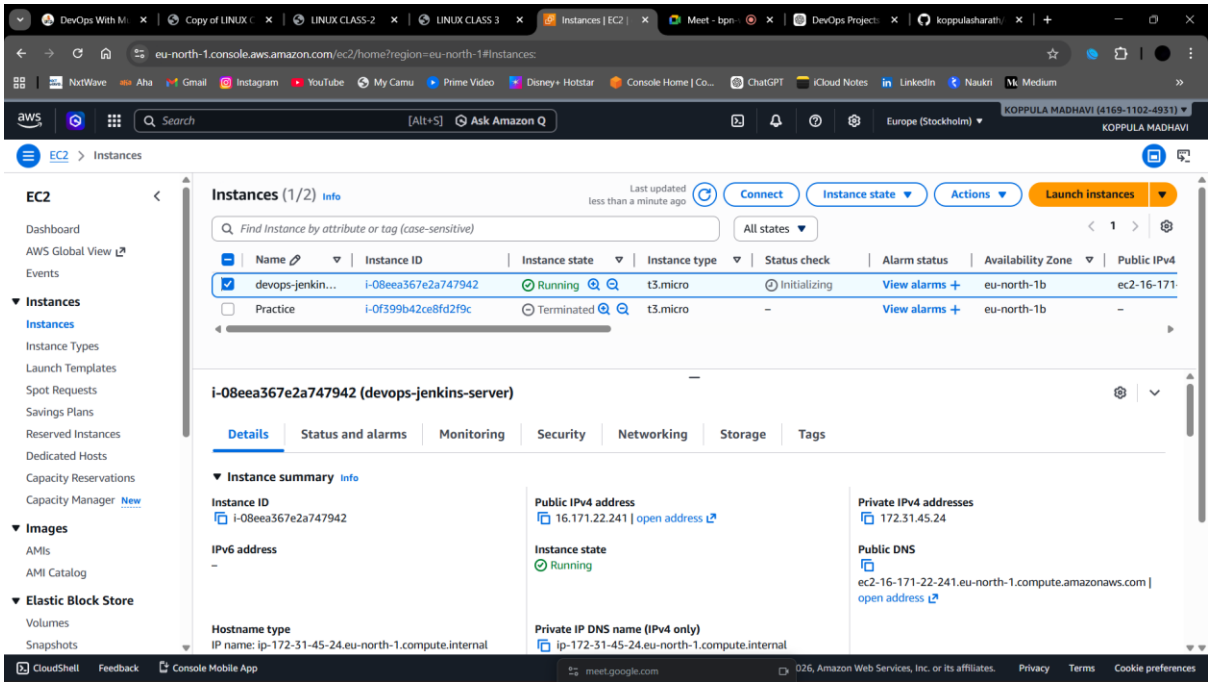
5. AWS EC2 Infrastructure

An EC2 instance was created in AWS cloud to host Jenkins and Docker.

Configuration:

- Instance Type: t3.micro
- Operating System: Amazon Linux
- Jenkins Port: 8080
- Application Port: 80

The EC2 server acts as the CI/CD build server where Jenkins executes the pipeline and deploys containers.



---

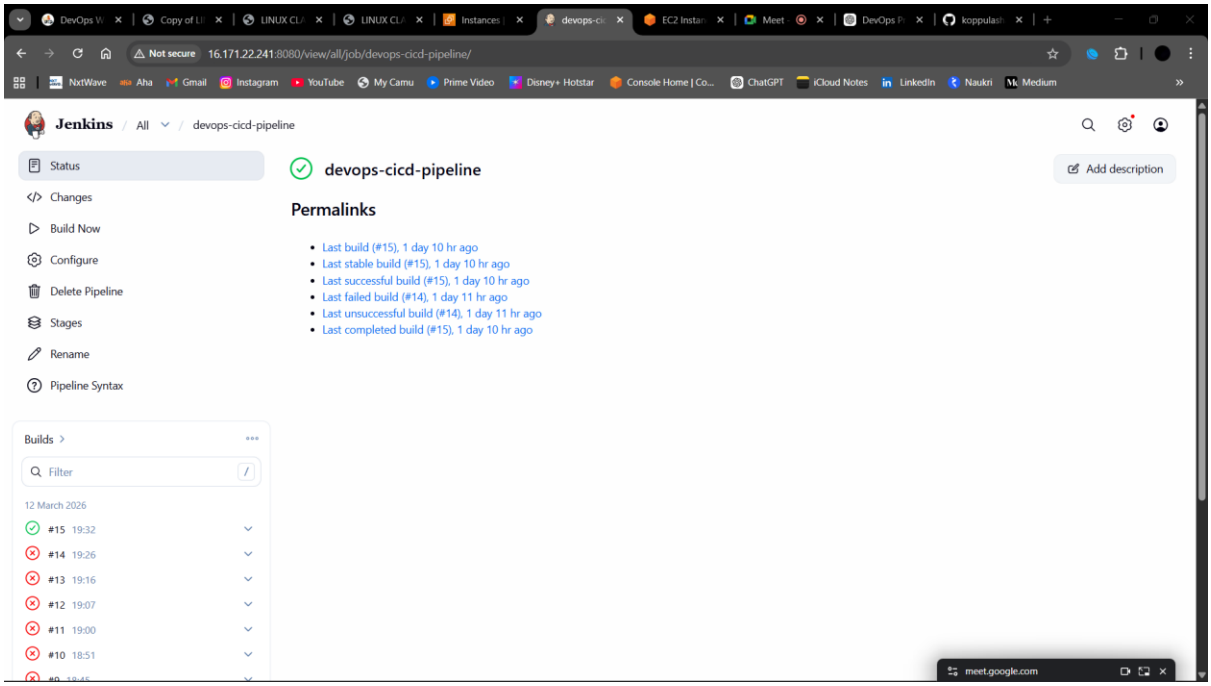
6. Jenkins Pipeline Configuration

A Jenkins pipeline job was created to automate application deployment.

Pipeline stages:

1. Clone Repository
2. Build Docker Image
3. Run Docker Container

Jenkins automatically fetches the source code from GitHub and executes the pipeline.



---

7. Jenkins Pipeline Execution Output

The Jenkins console output shows successful pipeline execution.

Started by user RIKKY

Stage 1: Clone Repository
Repository: https://github.com/koppulasharath/devops-cicd-pipeline.git
Branch: main

Stage 2: Build Docker Image
docker build -t devops-app .

Docker image successfully built.

Stage 3: Run Docker Container
docker run -d -p 80:3000 devops-app

Container ID:
465c3b41b4d7520f1a1148441af2641583600ba6c5478541019cff5bd79dcfef

Pipeline Result:
Finished: SUCCESS

This confirms the CI/CD pipeline executed successfully.

---

8. Docker Container Deployment

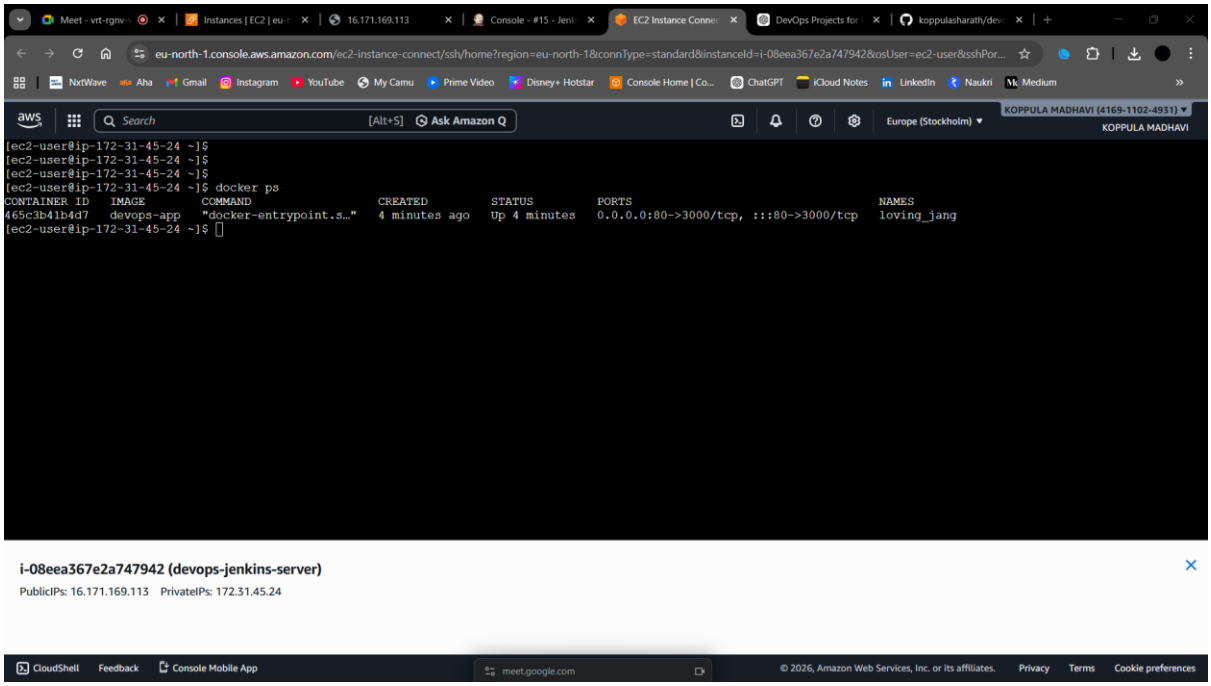
The container was verified using:

docker ps

Output shows:

devops-app container running
PORTS: 0.0.0.0:80 → 3000

This confirms that the application container is running successfully.



---

9. Application Deployment Result

The application was accessed using the EC2 public IP address.

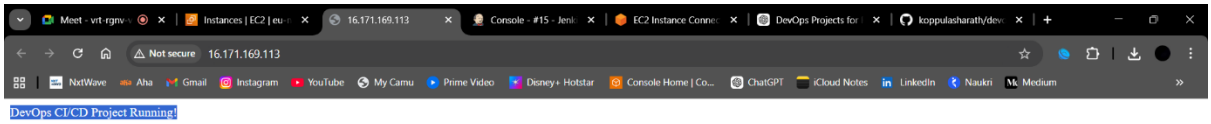
Example:

http://16.171.xx.xxx

The web browser displayed:

DevOps CI/CD Project Running!

This confirms the successful deployment of the application.



---

10. Linux Command Execution

During the setup process, the following commands were executed on the EC2 instance:

```
sudo yum update -y
sudo yum install docker -y
sudo systemctl start docker
sudo systemctl enable docker

sudo yum install java-17-amazon-corretto -y

sudo yum install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins

sudo yum install git -y

docker ps
```

These commands installed and configured Jenkins, Docker, Java, and Git on the EC2 server.

---

11. Project Outcome

The CI/CD pipeline successfully automated the following processes:

- Cloning application source code from GitHub
- Building Docker image automatically
- Deploying the application container

- Running the application on AWS EC2

This project demonstrates a practical DevOps workflow used in real-world environments.

---

12. Conclusion

This project implemented a complete DevOps CI/CD pipeline using Jenkins, Docker, GitHub, and AWS EC2.

The automated pipeline improves deployment efficiency, reduces manual errors, and ensures faster software delivery.

This project reflects real-world DevOps practices used by modern development teams.